Nested Dropdown / Mega Menu

Problem: Build an e-commerce style mega menu (think Amazon / Walmart "Shop by category"): a top bar of departments, each opening a large panel with a category list on the left and grouped links for the active category on the right.
Tech stack: React (Vite, JavaScript) (default)
Time limit: 60 minutes (default)
Often asked at: Amazon, Walmart
Why it matters: hover intent, accessibility

PART A: Thought process

Steps that work for any machine coding task:

1. **Clarify** – restate the problem, ask questions, write down assumptions.
2. **List interactions** – every user action and what must happen. This is your test checklist.
3. **Sketch components** – who renders what, who owns which state.
4. **Model state and data** – smallest state that can describe every screen; derive the rest.
5. **Spot the hard parts early** – decide how you will solve them before you type.
6. **Build in layers** – a working static version first, then one behavior at a time, always runnable.
7. **Cover states and edge cases** – empty, error, rapid input, keyboard, touch.
8. **Wrap up** – trade-offs and what you would do with more time.

Time budget for 60 minutes

Phase	Time
Requirements, questions, assumptions	5 min
Use cases + component / state design	8 min
Static markup + data rendering	10 min
Open/close (click, hover intent, outside click)	12 min
Keyboard + ARIA	12 min
Menu aim (diagonal mouse)	8 min
Edge cases, styling pass, wrap-up	5 min

Tip: think out loud. Say *why* you pick a delay, an ARIA pattern or a state shape. Interviewers grade reasoning, not keystrokes. If you run short on time, say what the next layer would be.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A horizontal bar of top-level items. Items with children open a mega panel; plain items are links.
- The panel shows a **category list** (left) and the **groups + links of the active category** (right).
- Opens on **hover** (desktop), **click / tap** (touch, mouse) and **keyboard**.
- Hovering another top item while a panel is open switches panels instantly.
- Closes on mouse leave (with a grace delay), **Escape**, **click outside**, and when **focus leaves** the menu.
- Hovering a category switches the right side; moving the mouse **diagonally** towards the right side must not switch categories on the way (menu aim).

Non-functional requirements

Area	What it means here
Performance	Only the open panel is rendered; no listeners on every link; timers cleaned up.
Scalability	Data driven: add a department or a category by editing JSON, not JSX.
Error handling	Missing children render a plain link; no crash on empty categories.
Accessibility	Disclosure pattern: `button` + `aria-expanded` + `aria-controls`; full keyboard; focus returns to trigger on Escape; visible focus ring.
Usability	Hover intent (no flicker when the cursor crosses the bar), close grace delay, menu aim, touch works without hover.
Reusability	`MegaMenu` takes `items` as a prop; timers live in a small `useTimeout` hook; menu-aim math is a pure function.
Extensibility	Data can come from an API later; the same tree can feed a mobile drawer.

Clarifying questions

1. Is the menu data static, or fetched from an API? Does it need lazy loading per department?
2. How deep is the tree? Fixed 3 levels (department → category → links) or arbitrary depth?
3. Must it work on touch devices and small screens (drawer / accordion)?
4. Should the top-level item itself be a link (navigate on click) or only open the panel?
5. What accessibility level is expected: keyboard only, or also screen reader announcements?
6. Any animation / reduced-motion requirement?

Assumptions we build with

- Static data in a local module, shape: department → categories → groups → links (3 levels below the bar).
- Top-level items with children are **buttons** (open panels); items without children are **links**.
- Desktop first; touch supported through click/tap; small screens stack the panel vertically.
- WAI-ARIA **disclosure navigation** pattern (not `role="menu"`), because the panel holds normal site links.
- Delays: open 150 ms, close 300 ms, menu-aim 300 ms.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Mouse rests on a top item ≥ 150 ms	Its panel opens, first category active
Mouse crosses the bar quickly	Nothing opens (pending open is replaced/cancelled)
Panel open, mouse moves to another top item	That panel opens instantly
Mouse leaves the menu	Panel closes after 300 ms; coming back within 300 ms keeps it open
Mouse hovers a plain link item (Today's Deals)	Any open panel closes after the grace delay
Mouse click on a trigger after hover opened it	Panel stays open (no toggle-close)
Tap on a trigger (touch)	Toggles the panel (no hover on touch)
Hover a category moving straight down	Right side switches immediately
Hover a category while moving diagonally to the right side	Switch is delayed; reaching the right side cancels it
Aim diagonally then stop on the category	It switches after 300 ms
`Enter` / `Space` on trigger	Toggles panel; `Tab` goes into the active category
`ArrowDown` on trigger	Opens panel and focuses the first category
`ArrowLeft` / `ArrowRight` on bar	Moves focus between top items (switches panel if one is open)
`ArrowUp` / `ArrowDown` / `Home` / `End` on categories	Moves focus and activates that category (wraps)
`ArrowRight` on category / `ArrowLeft` in links	Jumps into first link / back to the active category
`Escape` anywhere in menu	Closes, focus returns to the trigger
`Tab` out of the menu	Panel closes
Click / tap outside	Panel closes

---

3. Design the architecture

Module	Responsibility
`App`	Page shell; passes `MENU` data to the menu
`MegaMenu`	Top bar, which panel is open, hover intent, click/touch, outside click, Escape, focus-out, bar keyboard
`MegaPanel`	One open panel: active category, menu aim, category/link keyboard
`useTimeout`	One cancellable timer (schedule replaces, unmount cleans up)
`menuAim.js`	Pure geometry: `pointInTriangle` , `isAimingAt`

Module	Responsibility
`data/menu.js`	The menu tree

```
App
└── MegaMenu  (openId)                          ← useTimeout (open/close delays)
    └── li per top item
        ├── button.mega-trigger / a (plain link)
        └── MegaPanel  (activeId)  only when open ← useTimeout (aim delay), menuAim.js
            ├── ul.mega-categories  → button per category
            └── div.mega-content    → section per group → links
```

Start with one component that renders the tree. Split `MegaPanel` out once the panel gets its own state (active category) and its own keyboard handling; split `useTimeout` once two components need the same timer logic.

---

4. Design the state / data model

State or field	Owner	Purpose
`openId`	`MegaMenu`	Which top item's panel is open (`null` = closed)
`activeId`	`MegaPanel`	Which category's links are shown
`timer` (ref)	`useTimeout`	Pending open / close / aim action
`moves` (ref)	`MegaPanel`	Last 3 mouse positions for menu aim
`triggerRefs` , `catRefs` (refs)	each component	Moving focus by keyboard

```
// MegaMenu
const [openId, setOpenId] = useState(null)

// MegaPanel (mounted only while open)
const [activeId, setActiveId] = useState(item.categories[0].id)
const moves = useRef([])
```

Senior notes

- **Derived, not stored:** `isOpen = openId === item.id` , `active = categories.find(c => c.id === activeId)` , `aria-expanded` – all computed in render.
- **Refs, not state:** mouse positions and timer ids change on every mouse move but never change what is drawn. Putting them in state would re-render on each `mousemove` .
- **Reset by unmount:** `MegaPanel` is only rendered while open, so `activeId` naturally resets to the first category next time. No reset effect needed.
- **Ownership:** `openId` is global to the bar (only one panel open at a time) so it lives in the parent; `activeId` is local to a panel.

---

5. Design the data structures and contracts

```
// data/menu.js – stable, globally unique ids (they become DOM ids for aria-controls)
export const MENU = [
  {
    id: 'electronics',
    label: 'Electronics',
    categories: [
      {
        id: 'mobiles',
        label: 'Mobiles',
        groups: [
          { id: 'mobiles-brands', label: 'Top brands',
            links: [{ id: 'mobiles-apple', label: 'Apple', href: '#mobiles/Apple' }] },
        ],
      },
    ],
  },
  { id: 'deals', label: "Today's Deals", href: '#deals' }, // no categories → plain link
]
```

Why stable ids: React keys stay correct when the list changes, and `aria-controls` / `aria-labelledby` need unique DOM ids.

Component contracts

Contract	Input	Output
`<MegaMenu items />`	`items: TopItem[]`	Renders the nav; no callbacks needed (links navigate)
`<MegaPanel item id />`	one `TopItem` with `categories`, a DOM id	The open panel
`useTimeout()`	–	`{ schedule(fn, ms), cancel() }`
`isAimingAt(prev, curr, rect)`	two `{x,y}` points, submenu `DOMRect`	`boolean`

If the data came from an API (mock contract):

```
GET /api/menu            → 200 [{ id, label, href? , hasChildren }]
GET /api/menu/:id        → 200 { id, categories: [...] }   (lazy, per department)
```

---

6. Trace the data flow

Mouse opens "Fashion", then picks "Women":

```
pointerenter on <li Fashion> (pointerType = mouse)
   ↓ openId is null → schedule(setOpenId('fashion'), 150ms)
   ↓ 150 ms pass with no pointerleave
setOpenId('fashion')  →  MegaMenu re-renders
   ↓ isOpen true → <MegaPanel item=Fashion> mounts, activeId = 'men'
   ↓ button aria-expanded="true", panel visible with Men's groups
mousemove on category list → moves.current keeps last 3 points
   ↓
mouseenter on "Women" button
   ↓ isAimingAt(oldest point, current point, content rect)?
   ├─ no  → cancel(); setActiveId('women') → right side shows Women
   └─ yes → schedule(setActiveId('women'), 300ms)
            ├─ mouse enters right side → cancel() → Men stays
            └─ mouse stays → after 300ms Women shows
pointerleave on <li> → schedule(setOpenId(null), 300ms) → panel unmounts
```

---

7. Important technical challenges

Challenge 1: Hover intent (flicker and accidental close)

Problem: Opening on raw `mouseenter` makes panels flash open while the cursor just crosses the bar to reach the search box. Closing on raw `mouseleave` closes the panel when the cursor slips 2 px outside on its way down.

Solution: One cancellable timer. Enter schedules open after 150 ms; leave schedules close after 300 ms; a new schedule replaces the old one, so "leave then enter again" cancels the close. When a panel is already open, switching top items is instant because the user is clearly browsing.

```
const onPointerEnter = (e, item) => {
  if (e.pointerType !== 'mouse') return
  if (openId) open(item.id)                              // already browsing
  else schedule(() => setOpenId(item.id), OPEN_DELAY)    // intent check
}
const onPointerLeave = (e) => {
  if (e.pointerType === 'mouse') schedule(() => setOpenId(null), CLOSE_DELAY)
}
```

The panel is a DOM child of the `<li>`, so `pointerleave` does not fire when moving from the trigger into the panel, even though the panel is positioned under the whole bar.

Challenge 2: Menu aim (diagonal movement)

Problem: The user hovers "Men", sees "Sneakers" on the right and moves diagonally. The path crosses "Women", which swaps the right side away before the cursor arrives.

Solution: Keep the last few mouse positions. When a new category is entered, check if the cursor is inside the triangle formed by the older position and the top-left / bottom-left corners of the right side. If yes, the

user is aiming at the panel: delay the switch; reaching the panel cancels it; stopping on the category lets it fire.

```
export function isAimingAt(prev, curr, rect, tolerance = 20) {
  if (!prev || !rect || (prev.x === curr.x && prev.y === curr.y)) return false
  const top = { x: rect.left, y: rect.top - tolerance }
  const bottom = { x: rect.left, y: rect.bottom + tolerance }
  return pointInTriangle(curr, prev, top, bottom)
}
```

It is pure, so it is unit-tested with plain `node:assert` (see the test file in section 11).

Challenge 3: Hover vs click vs touch

Problem: (a) Mouse hover opens the panel at 150 ms; the user then clicks the trigger, and a naive toggle closes it. (b) On touch, a tap fires emulated `mouseenter` **and** `click`: open then immediately toggle shut.

Solution: Use Pointer Events. Hover handlers ignore anything that is not `pointerType === 'mouse'`. On click, a mouse click always *opens*; touch and keyboard (pointerType `''`) *toggle*.

```
const onTriggerClick = (e, id) => {
  if (e.nativeEvent.pointerType === 'mouse') open(id)
  else if (openId === id) close()
  else open(id)
}
```

Challenge 4: Accessibility and focus management

Problem: Mega menus are often mouse-only: `div`s with `onMouseEnter`, no keyboard, `role="menu"` misused for site links, focus lost after Escape.

Solution:

- **Disclosure pattern**: real `<button aria-expanded aria-controls>` triggers and real `<a>` links. `role="menu"` is for application menus (like a File menu) and forces screen readers into a mode where normal link navigation breaks.
- **Roving tabindex** in the category list: only the active category has `tabIndex=0`, so `Tab` goes from the active category straight into its links.
- **Escape** closes and returns focus to the trigger. **Focus out** (`onBlur` with `relatedTarget` outside the nav) closes.
- **ArrowDown** opens and focuses the first category; `flushSync` renders the panel synchronously so the button exists before we focus it.

```
const openAndFocusFirst = (id) => {
  flushSync(() => open(id))
  document.getElementById(`${id}-panel`)?.querySelector('button')?.focus()
}
```

---

8. Implement in the correct order

#	Step	Working result
1	Data file + render the bar from `MENU`	Static bar
2	`openId` state, click toggles, render panel with first category	Click to open
3	Render categories + groups/links, `activeId` on hover / focus	Browsable panel
4	Layout CSS: full-width panel under the bar, two columns	Looks like a mega menu
5	Escape, click outside, focus-out close	Closes properly
6	`useTimeout` + hover intent (open/close delays, instant switch)	Smooth mouse UX
7	Pointer types: hover only for mouse, click opens vs toggles	Touch + click-after-hover fixed
8	ARIA: `aria-expanded`, `aria-controls`, labelled sections, focus ring	Screen reader friendly
9	Keyboard: bar arrows, ArrowDown open, category arrows, roving tabindex, Left/Right jump	Full keyboard
10	Menu aim: track moves, `isAimingAt`, delayed switch, cancel on content enter	Diagonal moves work
11	Small screen media query, reduced motion	Responsive
12	Unit test for `menuAim.js`, manual test of the use-case table	Verified

---

9. Cover all states and edge cases

State / edge case	Handling
Initial	All closed, `aria-expanded="false"`
Opening (hover pending)	Timer pending; leaving within 150 ms cancels
Open	One panel, first category active
Closing (grace)	Timer pending; re-entering cancels
Empty (item has no categories)	Rendered as a plain `<a>`; hovering it closes any open panel
Category with one group / few links	Content wraps; layout keeps min height
Loading / error (if data is fetched)	Show skeleton in panel / fall back to top-level links only
Rapid hover across the bar	Each enter replaces the timer; nothing flickers
Rapid hover across categories while aiming	Only the last pending switch survives
Click after hover	Stays open
Touch tap	Toggles; emulated mouse events ignored
Escape	Closes, focus back on trigger

State / edge case	Handling
Tab out / click outside	Closes
Clicking empty panel area	Panel has `tabIndex=-1`, so focus stays inside and the blur handler does not close it
Unmount with timer pending	`useTimeout` clears it on unmount
Small screen	Panel becomes static, columns stack
Reduced motion	Fade-in only when `prefers-reduced-motion: no-preference`

10. Final structure

```
src/
├── App.jsx
├── App.css
├── main.jsx
├── data/
│   └── menu.js
├── hooks/
│   └── useTimeout.js
├── utils/
│   ├── menuAim.js
│   └── menuAim.test.mjs      (node src/utils/menuAim.test.mjs)
└── components/
    ├── MegaMenu.jsx
    └── MegaPanel.jsx
```

11. Full working code

`src/data/menu.js`

```
// Tree: top item -> categories -> groups -> links. Ids are globally unique (used for DOM ids).
const links = (prefix, names) =>
  names.map((name) => ({ id: `${prefix}-${name.toLowerCase().replace(/\W+/g, '-')}`, label: name, href:
`#${prefix}/${name}` }))

export const MENU = [
  {
    id: 'electronics',
    label: 'Electronics',
    categories: [
      {
        id: 'mobiles',
```

```
        label: 'Mobiles',
        groups: [
          { id: 'mobiles-brands', label: 'Top brands', links: links('mobiles', ['Apple', 'Samsung',
'OnePlus', 'Google Pixel']) },
          { id: 'mobiles-price', label: 'By price', links: links('mobiles-price', ['Under 10k', '10k -
20k', '20k - 40k', 'Above 40k']) },
        ],
      },
      {
        id: 'laptops',
        label: 'Laptops',
        groups: [
          { id: 'laptops-type', label: 'Type', links: links('laptops', ['Gaming', 'Thin & light', '2-in-
1', 'Chromebook']) },
          { id: 'laptops-brands', label: 'Brands', links: links('laptops-brands', ['Dell', 'HP',
'Lenovo', 'Asus']) },
        ],
      },
      {
        id: 'audio',
        label: 'Audio',
        groups: [{ id: 'audio-type', label: 'Type', links: links('audio', ['Earbuds', 'Headphones',
'Speakers']) }],
      },
    ],
  },
  {
    id: 'fashion',
    label: 'Fashion',
    categories: [
      {
        id: 'men',
        label: 'Men',
        groups: [
          { id: 'men-clothing', label: 'Clothing', links: links('men', ['T-shirts', 'Shirts', 'Jeans'])
},
          { id: 'men-footwear', label: 'Footwear', links: links('men-footwear', ['Sneakers', 'Formal',
'Sandals']) },
        ],
      },
      {
        id: 'women',
        label: 'Women',
        groups: [
          { id: 'women-clothing', label: 'Clothing', links: links('women', ['Dresses', 'Tops',
'Kurtas']) },
          { id: 'women-bags', label: 'Bags', links: links('women-bags', ['Handbags', 'Backpacks']) },
```

```
        ],
      },
    ],
  },
  {
    id: 'home',
    label: 'Home & Kitchen',
    categories: [
      {
        id: 'kitchen',
        label: 'Kitchen',
        groups: [{ id: 'kitchen-appliances', label: 'Appliances', links: links('kitchen', ['Mixers',
'Air fryers', 'Kettles']) }],
      },
      {
        id: 'furniture',
        label: 'Furniture',
        groups: [{ id: 'furniture-rooms', label: 'By room', links: links('furniture', ['Living',
'Bedroom', 'Office']) }],
      },
    ],
  },
  { id: 'deals', label: "Today's Deals", href: '#deals' },
]
```

src/utils/menuAim.js

```
// Sign of the cross product tells which side of edge (b -> c) point a lies on.
const side = (a, b, c) => (a.x - c.x) * (b.y - c.y) - (b.x - c.x) * (a.y - c.y)

export function pointInTriangle(p, a, b, c) {
  const d1 = side(p, a, b)
  const d2 = side(p, b, c)
  const d3 = side(p, c, a)
  const hasNeg = d1 < 0 || d2 < 0 || d3 < 0
  const hasPos = d1 > 0 || d2 > 0 || d3 > 0
  return !(hasNeg && hasPos)
}

// Menu aim: is the cursor moving from `prev` to `curr` towards the submenu on the right?
// True when `curr` is inside the triangle (prev, submenu top-left, submenu bottom-left).
export function isAimingAt(prev, curr, rect, tolerance = 20) {
  if (!prev || !rect || (prev.x === curr.x && prev.y === curr.y)) return false
  const top = { x: rect.left, y: rect.top - tolerance }
  const bottom = { x: rect.left, y: rect.bottom + tolerance }
```

```
  return pointInTriangle(curr, prev, top, bottom)
}
```

`src/utils/menuAim.test.mjs`

```
import assert from 'node:assert/strict'
import { pointInTriangle, isAimingAt } from './menuAim.js'

const tri = [{ x: 0, y: 0 }, { x: 10, y: 0 }, { x: 0, y: 10 }]
assert.equal(pointInTriangle({ x: 2, y: 2 }, ...tri), true)
assert.equal(pointInTriangle({ x: 9, y: 9 }, ...tri), false)
assert.equal(pointInTriangle({ x: 0, y: 5 }, ...tri), true) // on an edge counts as inside

// Submenu sits at x=200, y=0..400; category list on the left.
const rect = { left: 200, top: 0, bottom: 400 }
assert.equal(isAimingAt({ x: 100, y: 100 }, { x: 120, y: 110 }, rect), true) // diagonal towards panel
assert.equal(isAimingAt({ x: 100, y: 100 }, { x: 100, y: 130 }, rect), false) // straight down the list
assert.equal(isAimingAt({ x: 100, y: 100 }, { x: 90, y: 100 }, rect), false) // moving away
assert.equal(isAimingAt({ x: 100, y: 100 }, { x: 100, y: 100 }, rect), false) // not moving
assert.equal(isAimingAt(null, { x: 1, y: 1 }, rect), false) // no history yet
console.log('menuAim: all checks passed')
```

`src/hooks/useTimeout.js`

```
import { useCallback, useEffect, useRef } from 'react'

// One cancellable timer: scheduling again replaces the pending callback.
export function useTimeout() {
  const timer = useRef(null)

  const cancel = useCallback(() => {
    clearTimeout(timer.current)
    timer.current = null
  }, [])

  const schedule = useCallback((fn, ms) => {
    clearTimeout(timer.current)
    timer.current = setTimeout(fn, ms)
  }, [])

  useEffect(() => cancel, [cancel])

  return { schedule, cancel }
}
```

`src/components/MegaMenu.jsx`

```
import { useEffect, useRef, useState } from 'react'
import { flushSync } from 'react-dom'
import { useTimeout } from '../hooks/useTimeout'
import MegaPanel from './MegaPanel'

const OPEN_DELAY = 150 // ignore a cursor just passing over the bar
const CLOSE_DELAY = 300 // forgive a cursor that slips out for a moment

export default function MegaMenu({ items }) {
  const [openId, setOpenId] = useState(null)
  const navRef = useRef(null)
  const triggerRefs = useRef({})
  const { schedule, cancel } = useTimeout()

  const open = (id) => {
    cancel()
    setOpenId(id)
  }

  const close = () => {
    cancel()
    setOpenId(null)
  }

  // Click / tap outside closes.
  useEffect(() => {
    if (!openId) return
    const onPointerDown = (e) => {
      if (!navRef.current.contains(e.target)) {
        cancel()
        setOpenId(null)
      }
    }
    document.addEventListener('pointerdown', onPointerDown)
    return () => document.removeEventListener('pointerdown', onPointerDown)
  }, [openId, cancel])

  const onPointerEnter = (e, item) => {
    if (e.pointerType !== 'mouse') return // touch has no hover; tap (click) handles it
    if (!item.categories) return schedule(() => setOpenId(null), CLOSE_DELAY)
    if (openId) open(item.id) // already browsing: switch instantly
    else schedule(() => setOpenId(item.id), OPEN_DELAY)
  }

  const onPointerLeave = (e) => {
```

```
    if (e.pointerType === 'mouse') schedule(() => setOpenId(null), CLOSE_DELAY)
  }

  const onTriggerClick = (e, id) => {
    // Mouse already opened it on hover: a click must not toggle it shut. Touch / keyboard toggle.
    if (e.nativeEvent.pointerType === 'mouse') open(id)
    else if (openId === id) close()
    else open(id)
  }

  const openAndFocusFirst = (id) => {
    flushSync(() => open(id)) // render the panel now so we can move focus into it
    document.getElementById(`${id}-panel`)?.querySelector('button')?.focus()
  }

  const onTriggerKeyDown = (e, index) => {
    const move = (step) => {
      const next = items[(index + step + items.length) % items.length]
      triggerRefs.current[next.id]?.focus()
      if (openId) setOpenId(next.categories ? next.id : null)
    }
    const item = items[index]
    const keys = {
      ArrowRight: () => move(1),
      ArrowLeft: () => move(-1),
      ArrowDown: () => item.categories && openAndFocusFirst(item.id),
    }
    if (!keys[e.key]) return
    e.preventDefault()
    keys[e.key]()
  }

  const onNavKeyDown = (e) => {
    if (e.key !== 'Escape' || !openId) return
    const id = openId
    close()
    triggerRefs.current[id]?.focus() // give focus back to where the user started
  }

  const onNavBlur = (e) => {
    if (!navRef.current.contains(e.relatedTarget)) close() // Tab moved focus out of the menu
  }

  return (
    <nav className="mega-nav" aria-label="Main" ref={navRef} onKeyDown={onNavKeyDown} onBlur=
{onNavBlur}>
      <ul className="mega-bar">
```

```
        {items.map((item, index) => {
          const isOpen = openId === item.id
          const panelId = `${item.id}-panel`
          return (
            <li key={item.id} onPointerEnter={(e) => onPointerEnter(e, item)} onPointerLeave=
{onPointerLeave}>
              {item.categories ? (
                <button
                  ref={(el) => (triggerRefs.current[item.id] = el)}
                  type="button"
                  className="mega-trigger"
                  aria-expanded={isOpen}
                  aria-controls={isOpen ? panelId : undefined}
                  onClick={(e) => onTriggerClick(e, item.id)}
                  onKeyDown={(e) => onTriggerKeyDown(e, index)}
                >
                  {item.label} <span aria-hidden="true">▾</span>
                </button>
              ) : (
                <a
                  ref={(el) => (triggerRefs.current[item.id] = el)}
                  className="mega-trigger"
                  href={item.href}
                  onKeyDown={(e) => onTriggerKeyDown(e, index)}
                >
                  {item.label}
                </a>
              )}
              {isOpen && <MegaPanel key={item.id} item={item} id={panelId} />}
            </li>
          )
        })}
      </ul>
    </nav>
  )
}
```

src/components/MegaPanel.jsx

```
import { useRef, useState } from 'react'
import { useTimeout } from '../hooks/useTimeout'
import { isAimingAt } from '../utils/menuAim'

const AIM_DELAY = 300

export default function MegaPanel({ item, id }) {
  // Panel mounts on open, so the active category resets to the first one every time.
```

```
  const [activeId, setActiveId] = useState(item.categories[0].id)
  const contentRef = useRef(null)
  const catRefs = useRef([])
  const moves = useRef([]) // last few mouse positions, oldest first
  const { schedule, cancel } = useTimeout()

  const active = item.categories.find((c) => c.id === activeId)
  const contentId = `${id}-content`

  const trackMouse = (e) => {
    moves.current = [...moves.current.slice(-2), { x: e.clientX, y: e.clientY }]
  }

  const hoverCategory = (catId, e) => {
    const rect = contentRef.current?.getBoundingClientRect()
    const aiming = isAimingAt(moves.current[0], { x: e.clientX, y: e.clientY }, rect)
    if (aiming) schedule(() => setActiveId(catId), AIM_DELAY) // still heading to the panel: wait
    else {
      cancel()
      setActiveId(catId)
    }
  }

  const focusCategory = (index) => {
    const count = item.categories.length
    const next = (index + count) % count
    setActiveId(item.categories[next].id)
    catRefs.current[next]?.focus()
  }

  const onCategoryKeyDown = (e, index) => {
    const keys = {
      ArrowDown: () => focusCategory(index + 1),
      ArrowUp: () => focusCategory(index - 1),
      Home: () => focusCategory(0),
      End: () => focusCategory(item.categories.length - 1),
      ArrowRight: () => contentRef.current?.querySelector('a')?.focus(),
    }
    if (!keys[e.key]) return
    e.preventDefault()
    keys[e.key]()
  }

  const onContentKeyDown = (e) => {
    if (e.key !== 'ArrowLeft') return
    e.preventDefault()
    catRefs.current[item.categories.findIndex((c) => c.id === activeId)]?.focus()
```

```
  }

  return (
    // tabIndex -1: clicking empty panel space keeps focus inside the nav, so the blur handler doesn't
close it
    <div className="mega-panel" id={id} tabIndex={-1}>
      <ul className="mega-categories" onMouseMove={trackMouse}>
        {item.categories.map((cat, i) => (
          <li key={cat.id}>
            <button
              ref={(el) => (catRefs.current[i] = el)}
              type="button"
              className={cat.id === activeId ? 'is-active' : ''}
              aria-expanded={cat.id === activeId}
              aria-controls={contentId}
              tabIndex={cat.id === activeId ? 0 : -1}
              onMouseEnter={(e) => hoverCategory(cat.id, e)}
              onFocus={() => setActiveId(cat.id)}
              onKeyDown={(e) => onCategoryKeyDown(e, i)}
            >
              {cat.label}
            </button>
          </li>
        ))}
      </ul>

      <div
        className="mega-content"
        id={contentId}
        ref={contentRef}
        onMouseEnter={cancel}
        onKeyDown={onContentKeyDown}
      >
        {active.groups.map((group) => (
          <section key={group.id} aria-labelledby={`${group.id}-title`}>
            <h3 id={`${group.id}-title`}>{group.label}</h3>
            <ul>
              {group.links.map((link) => (
                <li key={link.id}>
                  <a href={link.href}>{link.label}</a>
                </li>
              ))}
            </ul>
          </section>
        ))}
      </div>
    </div>
```

```
  )
}
```

`src/App.jsx`

```
import MegaMenu from './components/MegaMenu'
import { MENU } from './data/menu'
import './App.css'

export default function App() {
  return (
    <>
      <header className="site-header">
        <strong className="logo">ShopKart</strong>
        <MegaMenu items={MENU} />
      </header>
      <main className="page">
        <h1>Nested dropdown / mega menu</h1>
        <p>Hover, click, tap or use the keyboard (Arrow keys, Tab, Escape) on the menu above.</p>
      </main>
    </>
  )
}
```

`src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

`src/App.css`

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; color: #1f2937; background: #f9fafb; }
ul { list-style: none; margin: 0; padding: 0; }

.site-header { position: relative; display: flex; align-items: center; gap: 24px; padding: 0 24px;
background: #131921; color: #fff; }
.logo { font-size: 20px; }
.page { padding: 24px; }
```

```
/* The bar is the positioning context so every panel spans its full width. */
.mega-nav { position: static; }
.mega-bar { display: flex; }
.mega-trigger {
  display: block; padding: 16px 14px; font: inherit; color: #fff; background: none;
  border: 0; cursor: pointer; text-decoration: none;
}
.mega-trigger:hover, .mega-trigger[aria-expanded='true'] { background: #232f3e; }
:focus-visible { outline: 3px solid #f59e0b; outline-offset: -3px; }

.mega-panel {
  position: absolute; top: 100%; left: 0; right: 0; z-index: 10;
  display: flex; min-height: 260px; background: #fff; color: #1f2937;
  box-shadow: 0 12px 24px rgb(0 0 0 / 0.15);
}
.mega-panel:focus { outline: none; }
.mega-categories { width: 220px; padding: 8px 0; border-right: 1px solid #e5e7eb; }
.mega-categories button {
  width: 100%; padding: 10px 20px; font: inherit; text-align: left;
  background: none; border: 0; cursor: pointer;
}
.mega-categories button.is-active { background: #fef3c7; font-weight: 600; }
.mega-content { flex: 1; display: flex; flex-wrap: wrap; gap: 32px; padding: 20px 28px; }
.mega-content h3 { margin: 0 0 8px; font-size: 14px; text-transform: uppercase; color: #6b7280; }
.mega-content a { display: block; padding: 4px 0; color: #1d4ed8; text-decoration: none; }
.mega-content a:hover { text-decoration: underline; }

/* Small screens: stack everything; tap toggles since there is no hover. */
@media (max-width: 768px) {
  .site-header { flex-direction: column; align-items: stretch; padding: 12px; }
  .mega-bar { flex-direction: column; }
  .mega-panel { position: static; flex-direction: column; min-height: 0; }
  .mega-categories { width: auto; border-right: 0; border-bottom: 1px solid #e5e7eb; }
}

@media (prefers-reduced-motion: no-preference) {
  .mega-panel { animation: fade-in 120ms ease-out; }
}
@keyframes fade-in { from { opacity: 0; } }
```

12. Wrap-up

Edge cases handled: cursor crossing the bar, slipping out of the panel, diagonal movement, click after hover, touch taps, Escape with focus return, Tab out, outside click, empty departments, timers on unmount, small screens, reduced motion.

Trade-offs to mention

- **Unmount when closed vs always in DOM (hidden):** unmounting keeps the DOM small and resets state for free, but links are not in the HTML for crawlers or `Ctrl+F`. For SEO, render all panels with the `hidden` attribute (or server-render the links) and toggle visibility.
- **Disclosure pattern vs `role="menubar"`:** disclosure is simpler and correct for site navigation; menubar gives arrow-key-only navigation but changes screen reader behavior and requires the full menu keyboard contract.
- **Fixed delays:** 150/300 ms are common values (and what jQuery hoverIntent / Amazon-style menu aim use), but not tuned per user. Could be props.
- **Fixed depth (3 levels):** simpler UI and code; arbitrary depth would need a recursive panel and flyouts.
- `flushSync`: small, contained use to focus newly rendered content; alternative is an `autoFocus` flag passed to the panel and a `useEffect`.

What to add with more time

- Tests: React Testing Library + `user-event` for keyboard flows; Playwright for real hover/aim timing.
- Lazy-load panel data per department on first hover intent (prefetch on `pointerenter`, show skeleton).
- Mobile drawer with drill-down (back button per level, like Amazon's hamburger menu).
- Collision handling for very wide/narrow viewports, max-height + scroll for long category lists.
- Analytics for which categories are opened; configurable delays via props.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why did you make the top items buttons instead of links?

A trigger that opens a panel is an action, so it is a `<button>` with `aria-expanded`. If the department must also navigate (e.g. "Electronics" landing page), add a "Shop all Electronics" link as the first item inside the panel instead of overloading one element with two jobs.

Q2. Why not `role="menu"` / `role="menubar"`?

ARIA menus are for application command menus. They put screen readers into a mode where links behave like menu items and require a strict arrow-key contract. WAI-ARIA recommends the **disclosure navigation** pattern for site nav, which is exactly buttons + `aria-expanded` + normal links.

Q3. What if the menu tree has arbitrary depth?

Make the panel recursive: a `MenuList` that renders items and, for items with `children`, a nested `MenuList` flyout positioned to the right. `activeId` becomes a path array (`['electronics','mobiles','apple']`), and each level's menu aim checks the next level's rect.

State design

Q4. Why is `activeId` in `MegaPanel` and not in `MegaMenu`?

Only the open panel cares about it, and it must reset each time a panel opens. Keeping it local plus unmounting on close resets it for free. Lifting it would need a reset effect and couple the bar to panel

internals.

Q5. Why store mouse positions in a ref instead of state?

They change on every `mousemove` (60+ times a second) but never change the rendered output. State would re-render the panel on every move for nothing; a ref is just a mutable box.

Q6. Could you use CSS `:hover` only, with no JS?

For the basic open, yes (`li:hover > .panel { display: block }`). But you lose hover intent delays, menu aim, touch handling, Escape, focus return and `aria-expanded` . `:focus-within` helps keyboard a bit, but it still cannot update ARIA state. CSS-only is fine as a no-JS fallback.

Hover intent

Q7. Why 150 ms open and 300 ms close? Why not 0?

0 ms open makes panels flash while the cursor crosses the bar; 0 ms close punishes a slightly inaccurate mouse path. ~100–200 ms is below what users notice as lag but above a passing cursor; close needs to be longer because people overshoot. I would make them props and tune with real usage data.

Q8. What happens if the user leaves and comes back within 300 ms?

`pointerleave` schedules the close; `pointerenter` calls `open()` (because `openId` is still set), which cancels that timer. One timer that is always replaced makes this race-free.

Q9. Why `pointerenter` and not `mouseenter` ?

Pointer Events tell us `pointerType` . On touch, browsers emulate `mouseenter` and `click` for a tap; with `mouseenter` the tap would open on hover then toggle closed on click. Ignoring non-mouse pointers in the hover handlers avoids that.

Menu aim

Q10. Explain menu aim in one sentence. Why a triangle?

If the cursor is inside the triangle formed by where it was a moment ago and the two left corners of the submenu, it is heading to the submenu, so we delay switching. The triangle is the set of straight paths from the old point into the submenu.

Q11. What if the user aims at the panel and just stops on the wrong category?

The switch is delayed, not dropped. After 300 ms the scheduled `setActiveId` fires, so the category under the cursor becomes active. If the cursor enters the content first, `cancel()` drops it.

Q12. What if the submenu opens to the left (RTL) or below?

`isAimingAt` takes the target rect; for RTL use `rect.right` corners, for a submenu below use the top-left/top-right corners. The triangle test itself does not change.

Implementation steps

Q13. How does focus move into the panel on ArrowDown when the panel is not rendered yet?

`flushSync(() => open(id))` makes React commit the panel synchronously, so the next line can find and focus the first category button. Alternative: pass a `focusFirst` flag and focus inside the panel's `useEffect`.

Q14. Why does clicking empty space in the panel not close it?

The nav closes on `blur` when `relatedTarget` is outside. Clicking a non-focusable `div` would make `relatedTarget` `null`. Giving the panel `tabIndex={-1}` makes it focusable by click, so focus stays inside the nav.

Q15. How do you close on outside click without leaking listeners?

Add a `pointerdown` listener on `document` only while a panel is open (`useEffect` depends on `openId`), and remove it in the cleanup. `pointerdown` (not `click`) closes before focus moves and works for touch.

Q16. What is roving tabindex and why use it for categories?

Only one item in a group is tabbable (`tabIndex=0`), the rest are `-1` and reached with arrow keys. Here it makes `Tab` go from the active category straight into its links instead of stepping through every category first.

Performance

Q17. What if there are 50 departments with 1,000 links each?

Only the open panel is mounted, so the DOM holds one panel at a time. Load each department's data lazily on hover intent (prefetch at `pointerenter`, before the 150 ms delay fires) and cache it. For very long category lists, add max-height + scroll; virtualization is rarely needed for menus.

Q18. What if the menu API is slow?

Render the bar from a small first payload, prefetch a department on hover intent, show a skeleton inside the panel while loading, and cache responses (memory or `sessionStorage`). On error show "Couldn't load, retry" in the panel; top-level links still work.

Q19. Does every `mousemove` re-render?

No. `trackMouse` writes to a ref. Re-renders happen only when `openId` or `activeId` actually changes.

Trade-offs

Q20. Why not a library (Radix NavigationMenu, Headless UI)?

In production I would likely use Radix `NavigationMenu`: it already handles disclosure ARIA, keyboard, delays and pointer types. In an interview the point is to show I understand what those libraries solve: hover intent, menu aim, focus management.

Q21. How would this work on mobile?

No hover, little width. Switch to a drawer: a hamburger button opens a full-height panel; each level slides in with a "Back" button (drill-down), or categories become an accordion. Same data tree, different presentation component.

Q22. How would you test it?

Unit-test `menuAim.js` with plain asserts (done). RTL + `user-event` for keyboard flows (ArrowDown focuses first category, Escape returns focus) using fake timers for delays. Playwright for real mouse paths to verify hover intent and menu aim, plus an axe accessibility scan.